From Hard Thresholds to Differentiable Binarization

The Mathematics of Real-Time Scene Text Detection (DBNet)

Nguyen Dang Long
vincentlalo-long

November 2026

Abstract

Real-time scene text detection requires converting continuous probability predictions into discrete bounding polygons. Conventional segmentation pipelines apply heuristic, hard thresholding during post-processing, severing gradient propagation and creating an uncalibrated train-test mismatch under visual distribution shifts. This paper provides a first-principles mathematical analysis of Differentiable Binarization (DBNet). We formalize text detection as dense binary prediction, examine why standard scalar thresholding and Straight-Through Estimators (STE) fail, and derive the exact forward and backward dynamics of the Differentiable Binarization operator. Furthermore, we analyze adaptive threshold map generation, distance-transform-based affinity map supervision, and polygon decoding geometry. Finally, we evaluate the theoretical failure modes, instance-level confidence calibration, and geometric trade-offs governing real-time scene text extraction.

1 Classical Semantic Segmentation: Principles and the Post-Processing Pipeline

1.1 Dense Pixel-Level Classification Formulation

In classical computer vision, dense segmentation frameworks (such as FCN [2], U-Net [3], and Mask R-CNN [4]) formulate object localization as a dense pixel-level classification task. Rather than directly predicting parametric bounding geometries (x, y, w, h, θ), the model is tasked with answering a localized hypothesis at every spatial coordinate (x, y): *"Does pixel (x, y) belong to an object instance or background scenery?"*

For scene text detection, the objective simplifies to dense binary segmentation over the spatial image lattice $\Omega = \{1, \dots, H\} \times \{1, \dots, W\}$:

Notation. Let $I \in \mathbb{R}^{H \times W \times 3}$ denote the input image; $p(x, y) \in [0, 1]$ is the model's predicted foreground probability at pixel (x, y); $y(x, y) \in \{0, 1\}$ is the ground-truth binary label; $P, Y \in [0, 1]^{H' \times W'}$ denote the corresponding full-lattice probability and label maps; $d((x, y), C)$ represents the Euclidean distance from (x, y) to a contour C; $\sigma(t) = 1/(1 + e^{-t})$ is the standard logistic sigmoid function; and $\mathbf{1}[\cdot]$ denotes the indicator function.

1.2 The Encoder–Decoder Paradigm: Receptive Field vs. Boundary Precision

To classify dense pixels without suffering prohibitive computational complexity $\mathcal{O}(H \times W)$ across deep feature spaces, architectures employ an asymmetric funnel topology:

1. **Encoder (Hierarchical Abstraction and Downsampling):** A convolutional backbone E_ϕ systematically reduces spatial dimensions through strided convolutions or max-pooling layers:

$$\mathbf{X}_0 = I \in \mathbb{R}^{H \times W \times 3} \xrightarrow{E_\phi} \mathbf{X}_l \in \mathbb{R}^{\frac{H}{2^l} \times \frac{W}{2^l} \times C_l}, \quad l \in \{1, \dots, L\}.$$

Each downsampling step expands the effective receptive field (ERF) at layer l:

$$R_l = R_{l-1} + (k_l - 1) \prod_{i=1}^{l-1} s_i, \tag{1}$$

where k_l and s_i denote kernel sizes and operational strides. While a large receptive field is essential for contextual semantic disambiguation (e.g., distinguishing text glyphs from complex background clutter), it incurs a severe loss of high-frequency spatial detail, blurring boundary edges and stroke corners.

2. **Skip Connections and Lateral Feature Fusion:** To recover lost spatial fidelity, architectures employ lateral skip pathways (e.g., channel concatenation in U-Net [3], or 1×1 projection combined with elementwise addition in Feature Pyramid Networks (FPN) [8]):

$$\mathbf{F}_l = \mathrm{Conv}_{3\times3}\Big(\mathrm{Proj}_{1\times1}(\mathbf{X}_l) + \mathrm{Upsample}_{2\times}(\mathbf{F}_{l+1}) \Big), \tag{2}$$

fusing high-level semantic abstractions with low-level, high-resolution edge features.

3. **Decoder (Spatial Resolution Recovery):** The decoder progressively reconstructs spatial dimensions via bilinear interpolation or transposed convolutions ($\mathcal{W}^\top * \mathbf{X}$), projecting the fused representation to an operating resolution $H' \times W'$ (typically $H/4 \times W/4$ for real-time throughput).

1.3 Dense Probability Modeling and Objective Formulations

At the terminal decoder layer, a 1×1 convolution coupled with a sigmoid activation emits a continuous probability map:

$$P = \sigma(f_\theta(I)) \in [0,1]^{H' \times W'}, \qquad p(x,y) = \frac{1}{1 + \exp\left(-f_\theta(I)(x,y)\right)}. \tag{3}$$

Supervision compares the continuous prediction P against ground-truth raster masks $Y \in \{0,1\}^{H' \times W'}$. Because text strokes typically occupy a negligible fraction of the image canvas ($\sum Y \ll H'W'$), standard cross-entropy suffers catastrophic class-imbalance collapse. Training therefore relies on composite loss objectives:

1. **Weighted / α-Balanced Cross-Entropy (BCE):**

$$\mathcal{L}_{\mathrm{BCE}}(P,Y) = -\frac{1}{N} \sum_{(x,y)} w(x,y) \Big[\alpha\, y \log p + (1-\alpha)(1-y) \log(1-p) \Big], \tag{4}$$

with $\alpha \in (0,1)$ (typically $\alpha = 0.7$, upweighting rare positive text pixels) and a spatial weight map $w(x,y) \geq 0$ designed to downweight ambiguous boundary bands.

2. **Soft Dice Loss (Continuous Set Intersection):** While BCE evaluates pixels independently, region-level losses optimize geometric set agreement. The soft Dice loss treats P and Y as fuzzy continuous sets [6]:

$$\mathcal{L}_{\mathrm{dice}}(P,Y) = 1 - \frac{2 \sum_{(x,y)} p(x,y)\, y(x,y) + \varepsilon}{\sum_{(x,y)} p(x,y) + \sum_{(x,y)} y(x,y) + \varepsilon}, \tag{5}$$

where $\varepsilon > 0$ prevents division by zero in all-background scenes; DBNet adopts this linear form (with binary labels $\sum y^2 = \sum y$, the squared variant of generalized Dice differs only in the prediction term). Differentiating with respect to $p(x,y)$, with $N = 2\sum p\, y + \varepsilon$ and $D = \sum p + \sum y + \varepsilon$,

$$\frac{\partial \mathcal{L}_{\mathrm{dice}}}{\partial p(x,y)} = \frac{N - 2\, y(x,y)\, D}{D^2}, \tag{6}$$

which penalizes each pixel in proportion to its overlap contribution versus the global region masses.

The segmentation objective combines both terms: $\mathcal{L}_{\mathrm{seg}} = \mathcal{L}_{\mathrm{dice}} + \mathcal{L}_{\mathrm{BCE}}$.

1.4 The Classical Post-Processing Pipeline: From Rasters to Geometric Instances

A fundamental limitation of traditional segmentation is that the neural network halts at the continuous raster map P. Extracting concrete instances (e.g., bounding boxes or bounding polygons) is offloaded entirely to a sequence of heuristic, external image-processing algorithms executed on the host CPU:

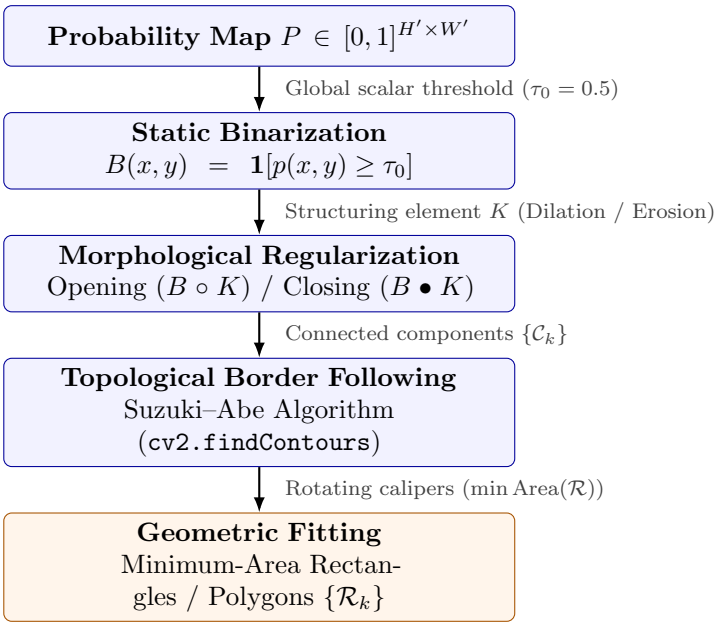


Figure 1: The classical multi-stage post-processing pipeline bridging continuous segmentation probabilities to discrete bounding instances.

The traditional pipeline executes in four sequential stages:

1. **Global Scalar Thresholding:** The continuous map P is converted into a binary mask $B(x, y)$ by applying a static scalar cutoff $\tau_0 \in (0, 1)$ (conventionally $\tau_0 = 0.5$):

$$B(x,y) = \mathbf{1}[p(x,y) \geq \tau_0] = \begin{cases} 1, & p(x,y) \geq \tau_0, \\ 0, & p(x,y) < \tau_0. \end{cases} \tag{7}$$

2. **Morphological Regularization:** Binary masks are filtered using structuring elements $K \in \{0,1\}^{m \times m}$ to eliminate isolated stroke artifacts via morphological opening $B \circ K = (B \ominus K) \oplus K$ and close internal stroke perforations via morphological closing $B \bullet K = (B \oplus K) \ominus K$.

3. **Topological Border Following (Suzuki–Abe Algorithm):** Connected components of foreground pixels ($B = 1$) are traced into vector boundary polygons using hierarchical border-following algorithms (Suzuki & Abe, 1985 [5]; standard `cv2.findContours`). The algorithm scans raster rows, identifies 0-to-1 boundary transitions, and traces 8-connected borders to emit ordered coordinate loops $\mathcal{C}_k = \{(x_i, y_i)\}_{i=1}^{M_k}$.

4. **Geometric Fitting:** For each traced contour $\mathcal{C}_k$, an enclosing geometric primitive is fitted:

$$\mathcal{R}_k^* = \arg\min_{\mathcal{R}} \text{Area}(\mathcal{R}) \quad \text{s.t.} \quad \mathcal{C}_k \subseteq \mathcal{R},$$

   typically solved via Freeman's rotating calipers algorithm to produce minimum-area bounding rectangles or oriented quadrilaterals.

1.5 The Scene Text Dilemma: Three Structural Breakdowns

When applied to scene text detection in video frames or complex natural scenes, this classical paradigm breaks down due to three fundamental structural failures:

1. **Text Clumping and Inter-Glyph Coalescence:** Typographic text features extremely dense character kerning, where the spatial spacing between consecutive glyphs Δx is often on the order of 1–3 pixels. Strided downsampling in the encoder and convolutional spatial smoothing blur these delicate gaps. Consequently, the inter-character background pixels exhibit ambiguous probabilities hovering in the range $p(x, y) \in [0.4, 0.6]$. Under a global threshold $\tau_0 = 0.5$, the binarizer bridges the gap between adjacent letters and adjacent lines, fusing entire words, phrases, or paragraphs into an intractable, monolithic connected component. Once merged into a single topological contour, downstream geometric fitting is powerless to separate individual text instances.

2. **Gradient Disconnection and Optimization Blindness:** The neural network is optimized purely for pixel-level likelihoods P, completely decoupled from the geometric quality of the resulting bounding boxes $\mathcal{R}_k$. The binarization step (7) and border-following operator are piecewise-constant step mappings whose classical derivatives with respect to P vanish almost everywhere:

$$\frac{\partial B(x, y)}{\partial p(x, y)} = \delta(p(x, y) - \tau_0) = 0 \quad \text{almost everywhere.} \tag{8}$$

   Because the gradient path is severed, errors committed during polygon fitting (e.g., loose boundaries, merged words, or truncated punctuation) cannot transmit supervisory gradient feedback to adjust the weights of the convolutional backbone. The network remains fundamentally blind to instance-level geometric errors.

3. **Computational Latency and Real-Time Infeasibility:** Morphological operations require $\mathcal{O}(H \times W \times |K|)$ serial pixel passes, and topological contour tracing (Suzuki–Abe) relies on sequential raster scanning with pointer-chasing memory patterns that cannot be parallelized on GPU or SIMD vector hardware. On high-resolution video streams (e.g., 720p or 1080p), serial CPU-bound post-processing consumes 50–200 ms per frame, single-handedly preventing the pipeline from achieving real-time throughput (> 30 FPS).

2 The Binarization Dilemma: From Heuristic Cuts to Differentiable Surrogates

2.1 Why Static Scalar Thresholding Fails Under Distribution Shift

A continuous probability map is not a terminal detection output: contours, polygons, and text lines are computed from a *binary* representation. The classical mapping operator is the Heaviside step function:

$$B(x, y) = \mathbf{1}[p(x, y) > \tau]\,, \tag{9}$$

for some scalar threshold τ. In classical edge detection, this formulation mirrors the Canny operator [1], which convolves with Gaussian derivatives G_σ, computes the gradient magnitude g, and thresholds g via a hysteresis pair (τ_ℓ, τ_h). The threshold is an unlearned, hand-tuned scalar.

When applied to deep neural probability maps, static scalar thresholding causes two catastrophic failure modes:

1. **Distribution shift under variable illumination and font geometry.** The empirical mean, dynamic range, and variance of $p(x, y)$ shift radically across distinct image samples due to variations in ambient lighting, motion blur, font weight, compression artifacts, and background contrast. A fixed scalar $\tau_0 = 0.5$ is fundamentally uncalibrated for any image whose posterior response does not align with the training distribution's empirical center. Conventional inference systems resort to ad-hoc, per-image or per-batch normalization ($p/\max(p)$)—an unlearned, fragile heuristic that fails under noisy outliers.

2. **Vanished gradients and optimization blockage.** The classical derivative of the Heaviside step is zero everywhere except at the singular boundary $p = \tau$:

$$\frac{\partial B}{\partial p} = \delta(p - \tau) = 0 \quad \text{for } p \neq \tau.$$

The mapping (9) completely discards all first-order gradient information regarding *how close* a borderline pixel lies to the decision boundary. While the neural network can be trained to minimize pixelwise loss on p, the downstream binarized output depends on a discontinuous function that backpropagation cannot penetrate.

The binarization gap in one sentence. Neural training optimizes a continuous probability field P; downstream geometry consumes a discrete binary lattice B; every bridge between the two that is omitted from the computational graph introduces an intractable train-test discrepancy.

2.2 Straight-Through Estimators: Bypassing, Not Solving

A widespread workaround adapted from quantized neural networks is the Straight-Through Estimator (STE) [7]. The STE executes the exact discontinuous step (9) during the forward pass, but substitutes an identity gradient operator during backpropagation:

$$\frac{\partial \mathcal{L}}{\partial p}\Big|_{\text{STE}} := \frac{\partial \mathcal{L}}{\partial B}. \tag{10}$$

While the STE prevents gradient starvation, the resulting gradient signal is fundamentally unfaithful to the underlying geometry:

1. It instructs every pixel to uniformly push p upward or downward regardless of whether p lies adjacent to the decision level set $p \approx \tau$ or deep within the interior of a background region.
2. Because the scalar τ does not participate in any differentiable mathematical expression, the STE provides no analytical mechanism to learn or adjust the threshold τ itself.

Differentiable Binarization (DBNet) [10] addresses both dilemmas in a unified, mathematically rigorous formulation: rather than forcing the network to adapt to an external heuristic threshold, the binarization operator is lifted directly into the differentiable computational graph.

3 Differentiable binarization (DB)

3.1 The DB layer

DB keeps the shape of (9)—a comparison between the probability map and a threshold—but makes two changes simultaneously:

1. the threshold τ becomes a *predicted, per-pixel map* $\tau(x, y)$ produced by a second convolutional branch;
2. the step function is replaced by a steep sigmoid, so the comparison is inside the computation graph.

The binarized output is

$$P^d = \sigma\big(\gamma(P - \tau)\big), \qquad \gamma \gg 1, \tag{11}$$

with γ a scale hyperparameter (order 10^3). Equation (11) is the entire method; everything below is analysis of its consequences.

3.2 It is still a binarizer

For any pixel with $p \neq \tau$,

$$\lim_{\gamma \to \infty} \sigma\big(\gamma(p - \tau)\big) = \mathbf{1}[p > \tau],$$

so (11) is a smooth one-parameter family converging pointwise to the hard decision of (9). At inference, implementations apply (11) with large γ and then threshold at 0.5, which by $\sigma(t) > 0.5 \iff t > 0$ is *exactly* the event $p > \tau(x, y)$. Train and inference compute the same decision; γ controls only the sharpness of the surrogate used for gradients, in the same way temperature controls the sharpness of a softmax.

3.3 Gradient analysis

With $s := P^d = \sigma(\gamma(p - \tau))$,

$$\frac{\partial s}{\partial t} = \gamma\, s(1 - s) \quad \text{for } t = p \text{ or } t = \tau,$$

hence by the chain rule

$$\frac{\partial \mathcal{L}}{\partial p} = \frac{\partial \mathcal{L}}{\partial s}\, \gamma\, s(1 - s), \tag{12}$$

$$\frac{\partial \mathcal{L}}{\partial \tau} = -\frac{\partial \mathcal{L}}{\partial s}\, \gamma\, s(1 - s). \tag{13}$$

Three properties explain why this works.

1. **Gradients concentrate on the level set.** The factor $s(1 - s)$ is maximal at $p = \tau$ and decays exponentially in $\gamma|p - \tau|$. Pixels that are certainly text or certainly background receive essentially no learning signal through the DB layer; the signal lives on pixels whose probability sits near the decision boundary. The DB layer thus performs a form of *learned level-set refinement*: the loss asks "where should the $p = \tau$ contour pass?" rather than "push every probability to zero or one".
2. **The threshold becomes trainable.** Equation (13) is a genuine derivative with respect to τ: any loss that prefers the boundary at a different location pushes the threshold branch to move $\tau(x, y)$ pixel by pixel. A scalar τ had no path to the loss under (9) or the STE.
3. **Opposite signs encode a geometry.** The gradients in (12) and (13) have equal magnitude and opposite sign: raising p or lowering τ both move s up. The network resolves them jointly, which is precisely the coupling that a fixed threshold cannot express.

3.4 The adaptive threshold map

The threshold branch is a small convolutional head on the same fused feature map X:

$$\tau' = \sigma(W_2 * \sigma(W_1 * X)) \in (0, 1)^{H' \times W'},$$

but the raw τ' is *not* the threshold used in (11). It is rescaled by the dynamic range of the segmentation branch's own feature response:

$$\tau(x, y) = \tau'(x, y) \cdot \max_{(u,v)} \sigma(f_\theta(I))\,(u, v). \tag{14}$$

Equation (14) is the calibration step that failure mode 1 of Section 2.1 demanded. If an image produces probabilities confined to $[0, 0.4]$, a threshold in $(0, 1)$ is meaningless; the multiplicative factor confines τ to the range actually reachable by P, while τ' decides *where inside that range* the level set passes, per pixel. The threshold is therefore content-adaptive in two independent senses: spatially (each pixel has its own τ) and per image (the range is re-normalized by (14)).

Summary. DB replaces "train a map, then threshold it with a constant" by "train a map and a threshold map, and make the thresholding itself a differentiable operation." The hard decision is recovered exactly at inference by the limit in Section 3.2.

4 The affinity map: geometric supervision of confidence

Segmentation supervision alone does not attach a scalar confidence to a whole text instance, which post-processing needs in order to reject false contours. DBNet learns an auxiliary *affinity map* S whose value encodes how deep a pixel lies inside the shrunk text region.

4.1 Text regions and distance transforms

Each text instance v is a region with boundary contour C_v and extent (h_v, w_v). Define the band of pixels associated with it,

$$V = \{(x, y) \mid d((x, y), C_v) \le R_v\}, \qquad R_v = \min(h_v, w_v) \cdot r,$$

where r is the *shrink ratio* (default $r = 0.5$). The shrunk kernel

$$\tilde{V} = \{(x, y) \in V \mid d((x, y), C_v) \le \tilde{r}\, R_v\}, \qquad \tilde{r} \in [0.5, 1.5],$$

is the inner core of the text stroke. The target affinity map is the piecewise distance field

$$\tilde{S}(x, y) = \begin{cases} 0, & (x, y) \in \tilde{V}, \\ d((x, y), C_v) - \tilde{r}\, R_v, & (x, y) \in V \setminus \tilde{V}, \\ 1, & (x, y) \notin V, \end{cases} \tag{15}$$

clipped to $[0, 1]$. Inside the kernel the target is zero, outside the band it is one, and the ring $V \setminus \tilde{V}$ carries a linear ramp measured by the distance transform. The affinity head (again a small conv on the fused features) is trained with pixelwise BCE,

$$\mathcal{L}_{\text{agg}} = \frac{1}{M} \sum_{(x,y)} \text{BCE}\Big(\tilde{S}(x, y),\, S(x, y)\Big).$$

4.2 Why shrink at all

The shrink ratio is the main geometric hyperparameter of the family and encodes a trade-off. Adjacent glyphs separated by a small gap merge into one blob when the un-shrunk region is used as the training target; supervising on the shrunk kernel $r < 1$ keeps characters separable and makes small text detectable without inflating false positives. At decoding, the shrunk region is expanded again along the distance transform by $\hat{d} = k\, R_v$ with the un-shrink factor $k = 1/r$ (default $k = 2$), recovering the original contour. Detection quality for tiny or dense text is governed by the (r, k) pair, not by the segmentation confidence.

4.3 Instance-level confidence

For a decoded contour $\hat{v}$ with shrunk region $\hat{V}$, the score is the mean predicted probability inside that core,

$$S_v = \frac{1}{|\hat{V}|} \sum_{(x,y) \in \hat{V}} p(x, y), \tag{16}$$

and contours with $S_v < T$ (or below a minimum area) are discarded. Equation (16) defines the instance-level detection confidence score. Its semantic interpretation requires precision: it measures the mean text-likeness (foreground probability) of the region kernel, rather than the geometric tightness of the enclosing contour outline or the lexical legibility of the underlying glyphs.

5 Full model, objective, and decoding

5.1 Architecture

A backbone (ResNet-50, or MobileNetV3 [9] for the lightweight variants used on CPU) feeds a feature pyramid (FPN) [8]; the multi-scale features are fused into X and then split into four heads sharing X:

- segmentation head $\to P$ (conv 3×3, conv 1×1, σ);
- threshold head $\to \tau'$, combined as (14);
- affinity head $\to S$;
- DB layer (11) followed by contour extraction.

The total objective is

$$\mathcal{L} = \mathcal{L}_{\text{seg}} + \beta\, \mathcal{L}_{\text{agg}}, \qquad \beta = 1.$$

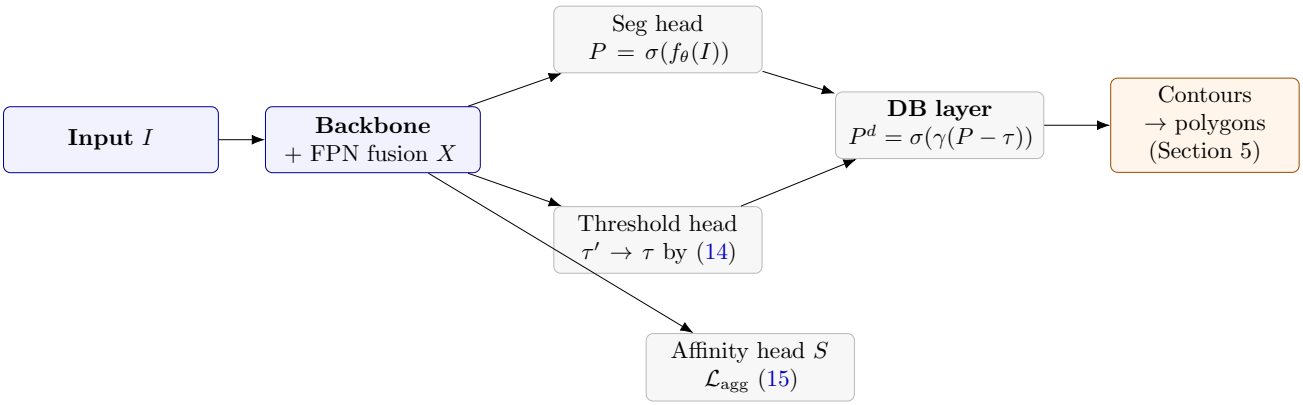


Figure 2: DBNet dataflow. The only novel component relative to a standard segmentation network is the DB layer and the threshold head; the affinity branch exists to supply instance-level geometry and confidence.

5.2 Decoding: from P^d to polygons

Inference is classical image processing on the binary map:

1. threshold P^d at 0.5 (equivalent to $p > \tau$, Section 3.2);
2. extract connected components / contours (Moore-neighbor tracing);
3. approximate each component by a minimum-area quadrilateral or polygon;
4. expand the shrunk region outward by $\hat{d} = kR_v$ along the distance transform to recover full glyph extents;
5. compute S_v by (16) and discard contours with $S_v < T$ or negligible area.

The emitted quadrilaterals are canonically sequenced in clockwise vertex order starting from the top-left coordinate: (top-left, top-right, bottom-right, bottom-left). This ordering is a convention of the min-area-rectangle decoder, not a mathematical guarantee of the detector, and it is precisely the convention that the Lyra artifact contract (`doc/SPECIFICATION.md`, Section 2) and `crop\_quad` rely on; a defensive vertex-order normalization before rectification is cheap insurance. The practical detector lineage of this project—the PP-OCR family [12, 13], whose detection head is a DB-formulation model trained with the objective above—emits exactly the `Detection.score` consumed by `detect.py`, whose semantics are Eq. (16).

6 What the construction buys, and what it does not

6.1 Comparison of binarization strategies

Table 1: Four ways to go from P to a binary map. Only DB keeps the decision inside the graph and learns the threshold.

Strategy	Forward decision	Backward signal	Failure mode
Fixed τ	$p > \tau_0$	none through decision	uncalibrated under distribution shift
Per-image norm + τ_0	$p > c \cdot \max(p)$	heuristic, unlearned	normalizer not in loss; brittle
STE	$p > \tau_0$	identity (incorrect)	gradient ignores level-set geometry; τ_0 frozen
DB (11)	$p > \tau(x, y)$	exact chain rule (12), (13)	extra branch; γ must be tuned; affinity design required

6.2 Boundary analysis and successors

Because the gradient of (11) lives on $|p - \tau| \lesssim 1/\gamma$, DB behaves like a level-set method whose boundary is moved by the task loss. This explains the family's behaviour on hard geometry: curved and irregular text benefits, and DBNet++ [11] extends the recipe in the same direction—a pixel-level threshold refinement (PIX) module that predicts a *signed* offset field to τ instead of a scalar rescale, plus a pixel-promote module (PPM) that feeds the binary result back into the features—making the level set adaptive at sub-pixel precision while staying boundary-free. The mathematical content is unchanged: the Heaviside of (9) is everywhere, and the question is only who gets to choose τ.

6.3 Theoretical Limitations and Geometric Edge Cases

1. **Score semantics.** The instance score S_v (16) quantifies region-level foreground probability, not geometric contour precision or character recognizability. Consequently, high-confidence detections can exhibit spatial boundary jitter on the order of several pixels across consecutive video frames, necessitating robust temporal tracking tolerance.
2. **Kernel scale sensitivity.** The shrink ratio r and un-shrink factor k define an intrinsic spatial resolution limit. Densely spaced characters or miniature typography with narrow kerning risk agglomerating into merged single-blob detections or under-segmenting into fragmented contours.
3. **Quadrilateral approximation bounds.** Minimum-area rectangle approximations are restricted to four vertices and degrade on severely curved, circular, or perspective-warped typography. Arbitrary-shape text requires unconstrained polygon decoding or explicit boundary-contour modeling.
4. **Semantic agnosticism toward distractors.** Because the adaptive threshold is guided strictly by visual contrast and geometric edge gradients, the detector remains inherently agnostic to semantic intent: persistent background logos, watermarks, and scene text are detected with equally high confidence as salient foreground text. Semantic discrimination cannot be resolved by spatial localization alone and must be deferred to temporal trajectory analysis.

7 Conclusion

DBNet's core theoretical contribution lies in embedding the decision boundary directly into the differentiable computational graph: the step function $\mathbf{1}[p > \tau]$ is approximated by $\sigma(\gamma(P - \tau))$ parameterized by a dynamically predicted, content-adaptive threshold map $\tau(x, y)$ normalized via (14). The resulting coupled gradients (12)–(13) actively optimize level-set contour placement rather than merely adjusting uncalibrated probability magnitudes. At inference, the exact hard binarization is

preserved, while auxiliary distance-transform affinity modeling (15) provides the geometric foundation for scored polygon reconstruction (16). Understanding this formulation clarifies both the structural capabilities of the DB architecture—differentiable boundary optimization, robust contrast adaptation, fast inference—and its intrinsic boundaries: score semantics measure text probability rather than semantic relevance, bounding rectangles suffer sub-pixel contour jitter, and semantic discrimination among visual text instances fundamentally requires downstream temporal reasoning.

References

[1] Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6), 679–698.

[2] Long, J., Shelhamer, E., & Darrell, T. (2015). Fully convolutional networks for semantic segmentation. *Proceedings of CVPR*, 3431–3440.

[3] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. *Proceedings of MICCAI*, 234–241.

[4] He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask R-CNN. *Proceedings of ICCV*, 2961–2969.

[5] Suzuki, S., & Abe, K. (1985). Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing*, 30(1), 32–46.

[6] Milletarí, J., Siebert, H., & Ewert, S. (2016). Fast automatic segmentation of the inner ear with 3D fully convolutional networks. *Proceedings of FIMH*, 118–127. (Soft Dice loss for region overlap; see also Sudre et al., 2017.)

[7] Bengio, Y., Lévine, N., & Scardapane, S. (2013). Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv:1308.3432*.

[8] Lin, T.-Y., Dollár, P., Girshick, R., He, K., & Hariharan, B. (2017). Feature pyramid networks for object detection. *Proceedings of CVPR*, 2117–2125.

[9] Howard, M., et al. (2019). Searching for MobileNetV3. *Proceedings of ICCV*, 1063–1072.

[10] Miao, M., Wu, B., Zhang, Y., Liao, H., Shen, J., & He, C. (2020). Real-time scene text detection with differentiable binarization. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(7), 11578–11585. arXiv:1911.08947.

[11] Liu, Y., Miao, Z., et al. (2022). DBNet++: A boundary-free framework with better shape and confidence for text detection. *arXiv:2202.10304*.

[12] Du, Z., Chen, X., Li, Y., Zhang, H., Hu, W., Mai, X., & Cui, C. (2020). PP-OCR: A practical OCR system. *arXiv:2006.03127*.

[13] Cui, C., Sun, C., Tian, X., Zheng, H., Zhou, X., Wei, H., Geng, Q., Jiang, K., Wei, S., Du, Z., & Li, Z. (2022). PP-OCRv4: Expanding the frontier of practical OCR systems without any cost. *arXiv:2208.07034*.